Truth Table to Circuit

A Research Trial Log

Stage 1: from exact construction to evidence-based research decisions

UFUG2106 Discrete Mathematics Project

Research record through Round 6.1 — 11 November 2026

Purpose. This document records the *path of reasoning*: what prompted each new algorithm, why the mathematical construction works, what the measurements show, and why the next experiment followed. It is intended for later self-study and transparent public research presentation. It is *not* a replacement for the shorter Stage 1 technical report or for the primary evidence ZIPs.

Version status: Stage 1 research record v1.0, backed by the frozen Round 5.1 research source and the archived experimental record.

Contents

1 How to read this record 2

2 Starting point: what is the real synthesis problem? 3

3 A recurring three-input, two-output example 4

4 Round 1: obtain an indisputably correct construction 5

5 Round 2: representation is an optimization variable 6

6 Round 3: account for complements, shared algebra and timing slack 7

7 Round 4: phase selection and algebraic rewriting 9

8 Round 4.1: construct the first explicit Area–Runtime Pareto comparison 10

9 Round 5: remove recomputation before reducing search quality 10

10 Round 5.1: treat the netlist as a data structure, not repeated text 11

11 Round 6.0: a change of representation, not merely another BDD order 13

12 Round 6.1: a deliberate Go/No-Go experiment 14

13 How We Constructed Non-Official Test Cases 16

14 How did we guard against benchmark overfitting? 16

15 A consolidated evidence narrative 16

16 How to reconstruct the evidence and figures 18

17 Reflection and next steps 18

18 Collaboration, AI-Assisted Development, and Responsibility 19

1 How to read this record

This log is intentionally more explanatory than a conventional technical paper. A reader need not know BDDs, implicants, or logic synthesis in advance. We begin with complete truth tables and the three allowed gates, then explain how each design limitation motivates the next technique. Whenever a mathematical identity is used, we distinguish that *the Boolean function is preserved* from the separate empirical claim that *the resulting circuit is cheaper or faster to synthesize*.

The research was not one straight sequence of uniformly better algorithms. Some changes reduced area at considerable runtime cost; a few optimization proposals were not retained. Consequently, the final architectural choice is not simply "the version with the smallest aggregate area." Round 5.1 is retained as the principal efficiency candidate, while Round 4.1 Strict and Rounds 6.0/6.1 provide informative alternative points on the engineering tradeoff.

1.1 Two reading routes

Readers who are new to logic synthesis should begin with the synthesis problem and the recurring three-input, two-output example, then follow Rounds 1–3 to see why representation and timing analysis are needed. Readers primarily interested in the engineering decisions can begin with the research map, examine the paired runtime and cut-rewriting results in Rounds 4.1–6.1, and finish with the evidence and reflection sections. Both routes rely on the same definitions and archived measurements; the shorter Technical Report presents the selected method rather than every development turn.

1.2 What the metrics do and do not mean

For a legal netlist G with gate-type areas a_t and delays d_t, the output-file area is

$$A(G) = \sum_{t \in \{\text{AND2},\text{OR2},\text{NOT}\}} N_t(G) a_t. \tag{1}$$

Every emitted gate is charged exactly once, even if its output fans out to several consumers. The network delay, in contrast, is a *critical-path* quantity:

$$T(v) = \max_{u \in \text{fanin}(v)} T(u) + d_{\text{op}(v)}, \quad D(G) = \max_{y \in \text{outputs}} T(y), \tag{2}$$

with source arrival zero (for NOT, the maximum has one argument). Finally, the synthesizer runtime T_{synth} is the wall-clock time of the *program constructing* G, not the propagation delay $D(G)$. These are dimensionally and conceptually different. The official evaluator additionally distinguishes the Boolean/format/limit status `PASS` from `target_met`. The provided project instructions and evaluator interface [1] do not reveal a numerical grading formula, so our totals across cases are engineering diagnostics rather than an official grade.

1.3 A short notation guide

The same symbols recur at different stages of the project. The distinction between the Boolean function and its implementation matters throughout:

Table 1: Notation used throughout the trial record. These are definitions, not performance claims.

Symbol	Meaning
F or f	Desired Boolean output vector or a single Boolean function from the input truth table.
G	Concrete, directed acyclic gate network realizing F.
$A(G)$	Charged area of all live gates in G (shared gates count once).
$D(G)$	Maximum output arrival time; a property of the circuit.
T_{synth}	Wall-clock runtime of the synthesis *program*, not the circuit delay.
$T(v), R(v)$	Arrival time and required arrival time of a node in a fixed network.
$\rho(G)$	The program's internal lexicographic candidate ranking, not an official score.

1.4 The frozen solver's ranking is not the evaluator's score

The Round 5.1 code uses a lexicographic tuple

$$\rho(G) = \big(\max(0, D - D_{\text{target}}),\ \mathbf{1}_{D \le D_{\text{target}}} A,\ D,\ A\big).$$

A feasible candidate is ranked by area and then delay; an infeasible candidate is first ranked by how badly it misses the target. This tuple is an *internal engineering heuristic*. The evaluator's public CLI exposes limits and performance metrics, but not a numerical grade formula. A clock-dependent search also means improving runtime can change which candidates finish before the cutoff; identical fixed-candidate semantics do not logically force identical final output bytes.

1.5 The narrative contract

Each iteration answers six questions: (1) What observable problem remained? (2) Which mathematical property suggested an answer? (3) What was implemented, rather than merely proposed? (4) What independent check supports correctness? (5) What happened to area, delay and runtime under measured conditions? (6) How did this evidence change the next decision? Formal results are statements about functions or algorithms under assumptions; benchmark claims are observations of frozen implementations.

1.6 The research map: decisions, not just versions

A conventional changelog would list features in implementation order. A research notebook should reveal why an observation made the next experiment reasonable. Table 2 summarizes that logic; each transition will later be examined with its precise mathematical condition and supporting evidence.

Table 2: A question-led map of the Stage 1 research path. "Observation" denotes the development evidence prompting the next question, not a controlled causal effect.

Move	Observation to explain	Next hypothesis
R1 → R2	Correct netlists can still miss delay targets.	Change decision orders and mapping forms, not the truth function.
R2 → R3	Better representations still leave costly shared gates and slack unused.	Optimize the costed DAG under global timing requirements.
R3 → R4	A local De Morgan choice ignores shared polarity across consumers.	Evaluate phase and shallow algebra jointly with real DAG costs.
R4 → R4.1	Additional candidates yield small area gains but expensive searches.	Make Area–Runtime tradeoffs explicit.
R4.1 → R5–5.1	Repeated structure building and text processing waste search time.	Preserve candidate semantics while eliminating recomputation.
R5.1 → R6	A fast mapping framework need not exhaust local Boolean alternatives.	Resynthesize bounded cuts under exact full-network costs.
R6.0 → R6.1	Valid area gains do not justify the measured runtime.	Cache and prioritize cuts with a fixed search budget; accept or stop.

What would have counted as failure? A proposed new mapping would be rejected if it broke functionality, missed required delay targets without a compensating purpose, or exhausted permitted resources. An engineering refactor would not qualify as an area-preserving speedup merely because it was faster on one selected case: it had to retain complete official results and survive paired and additional-function checks. These tests do not prove hidden-case performance, but they make the research decisions falsifiable.

2 Starting point: what is the real synthesis problem?

A complete truth table for $F : \{0,1\}^n \to \{0,1\}^m$ contains 2^n input assignments and m output bits per row. One can always write a sum of minterms: for each row on which $f = 1$, form the conjunction of the required input literals, then OR all such conjunctions. The construction is exact, but it can repeat the same combinations, use many NOT gates, and produce long chains of ANDs and ORs. This shows the central tension: *existence of a legal circuit* is elementary, while finding a good shared circuit is not.

The allowed gate set is not a library of all Boolean operations. A multiplexer, XOR, or majority gate can appear in an abstract derivation but must be lowered to AND2, OR2, and NOT, each priced by the current test case. For example,

$$a \oplus b = (a \vee b) \wedge \neg(a \wedge b). \tag{3}$$

This identity proves implementability, but not that its right-hand side has optimal area or delay. Gate sharing and heterogeneous prices determine which realization is preferable. A gate appearing in two output expressions should ideally be computed once and reused; however, choosing what to share can affect the critical path.

The frozen Round 5.1 source reads the optional runtime limit but does not explicitly parse or enforce a separately supplied ninth-line RAM limit. The evaluator may enforce memory externally; a successful published run or one Peak

RSS sample is not a universal guarantee. The full truth table alone requires reading $\Omega(m2^n)$ input bits, and ROBDD size and heuristic search costs can be exponential in the worst case.

A second early realization is that the optimum is constrained rather than scalar. The natural question is whether we can reduce $A(G)$ while keeping $D(G) \le D_{\text{target}}$ and avoiding program limits. We will repeatedly return to this distinction.

Four levels of objects: keep them separate

The *truth table* is the specification of F. A *BDD* is one compact way to represent that function by recursive decisions. A *gate DAG* is the actual implementation with fanout, prices, and timing. Finally, the *synthesizer* is the program searching for such DAGs and consuming CPU time and memory. A theorem about one level must not silently be turned into a performance claim about another.

3 A recurring three-input, two-output example

This running example will distinguish Boolean reasoning, structural representation and priced implementation throughout the log. Let

$$f_1 = (a \wedge b) \vee (\neg a \wedge c), \qquad f_2 = f_1 \vee b.$$

The complete table, ordered by (a, b, c), is:

a	b	c	f_1	f_2
0	0	0	0	0
0	0	1	1	1
0	1	0	0	1
0	1	1	1	1
1	0	0	0	0
1	0	1	0	0
1	1	0	1	1
1	1	1	1	1

Shannon step. Restriction at $a = 0$ gives $f_1 = c$ and $f_2 = b \vee c$; restriction at $a = 1$ gives $f_1 = f_2 = b$. Therefore the two output BDDs share the b cofactor. Yet a shared *BDD node* is not a physical gate, and shared gate placement must be decided after mapping.

To make the reduction visible, the root decisions can be written as

	$a = 0$	$a = 1$
f_1	c	b
f_2	$b \vee c$	b

.

One decision graph with root a for each output therefore points to the *same* residual subproblem b in both high branches. The equality of two cofactors is a mathematical reason for merging them; no heuristic guess is involved. It does *not* imply that a post-mapping hardware node has to be reused at every possible location.

A legal netlist. Using ID 0 for constant zero and IDs 1, 2, 3 for a, b, c, take

ID	gate and inputs
4	NOT(1)
5	AND2(1, 2)
6	AND2(4, 3)
7	OR2(5, 6)
8	OR2(7, 2)

$(y_1, y_2) = (7, 8).$

For illustrative areas $a_{\text{AND2}} = 2$, $a_{\text{OR2}} = 3$, $a_{\text{NOT}} = 1$, the area is $2(2) + 2(3) + 1 = 11$. Let the respective gate delays be $2, 2, 1$. Forward arrival propagation gives $T(4) = 1$, $T(5) = 2$, $T(6) = 3$, $T(7) = 5$ and $T(8) = 7$, hence network delay 7. With target 8, $R(8) = 8$ and $R(7) = \min\{8, 8 - 2\} = 6$, so gate 7 has slack $R(7) - T(7) = 1$ along the path to output 8. Although 7 is itself the first primary output, it is also a fanin of gate 8: its required time is therefore *stricter than the nominal output target*. This provides a concrete example of why multi-output sharing matters in the backward timing recurrence.

A useful surprise. Absorption changes f_2 to $b \vee (\neg a \wedge c)$. Gate 8 can therefore use inputs $(2, 6)$ without changing any other line, producing the same two Boolean outputs and the same area, but output f_2 then arrives at time 5. This is not a global optimum proof; it is a controlled illustration of why algebra, fanout and timing must be considered jointly.

What happens to required time after a rewrite? After absorbing f_2 into the direct gate $8 = \mathrm{OR2}(2, 6)$, gate 7 no longer feeds gate 8, although it remains the first primary output. Its *new* required time becomes $R'(7) = 8$ rather than 6. Thus a valid rewrite can change the fanout graph and the required-time constraints even when it preserves both Boolean outputs. This is exactly why old required times are local proof tools for a *fixed-boundary* rewrite, not numbers that may safely be reused after arbitrary restructuring.

4 Round 1: obtain an indisputably correct construction

4.1 Observation and motivation

Before comparing elaborate heuristics, we need a constructor that never relies on guessing hidden circuit structure. A complete table does not disclose the network that produced it, and many distinct networks can implement exactly the same function. The logical target should therefore be *equivalence to the table*, not reconstruction of a hypothetical unique original gate diagram.

4.2 The mathematical step: Shannon decomposition

Pick any variable x_i and restrict the function to its two possible values: $f_0 = f|_{x_i=0}$ and $f_1 = f|_{x_i=1}$. The Shannon identity, a foundation of switching-network synthesis [2], is

$$f = \neg x_i \wedge f_0 \ \vee \ x_i \wedge f_1. \tag{4}$$

Why is this true? If $x_i = 0$, the first conjunction reduces to f_0 and the second to zero; if $x_i = 1$, the roles exchange. These two cases exhaust the domain. Repeatedly applying (4) reduces every function to constants at the leaves and is therefore a constructive correctness argument.

But a full Shannon *tree* may represent the same cofactor in many branches. Consider a function whose truth values no longer depend on a variable after earlier decisions; continuing to branch on that variable is redundant. Another possibility is that two different branches lead to the identical remaining function. These are representation-level redundancies, not exotic Boolean identities.

4.3 Reduced Ordered Binary Decision Diagrams

Fix a variable order π. A BDD node stores its decision variable and the references to its low and high cofactors. We apply two exact reductions: (i) if the two child references are equal, skip that decision; (ii) if an identical node with the same variable and child references exists, reuse it from a unique table. The result is a shared DAG, not merely a binary tree. Bryant's ROBDD framework explains why fixed ordering and reduction give a canonical function representation [3].

Theorem 1 (Structural correctness of reduced Shannon DAGs)**.** *If every terminal has the correct constant value and every decision node computes (4), merging nodes of equal triples and deleting nodes with equal children leaves every output function unchanged.*

Proof. The full decision expansion computes the table by repeated case analysis. Merging two identical triples merges identical decision expressions. When the low and high child coincide, $\neg x f_0 \vee x f_0 = f_0$, so the parent is redundant. Apply these identities at each reduction step. □

4.4 Implementation and evidence

The baseline lowers decisions to the allowed physical gates, shares equal gate expressions via structural hashing, and outputs topologically ordered IDs. The Round 1 WSL evidence is 25/25 official PASS but only 12/25 cases meeting the target delay; summed area is 1,103,455. The independent checker additionally verified the outputs under its own exhaustive-or-sampled procedure.

Decision after Round 1

Learning: the algorithm is correct, but correctness alone says nothing about whether its variable order, sharing pattern and gate depth are good. Next, keep the exact Shannon foundation but widen the space of equivalent realizations.

5 Round 2: representation is an optimization variable

5.1 Why a smaller BDD need not imply a smaller circuit

A BDD is an abstract decision representation; the assignment asks for a priced AND/OR/NOT netlist. Two BDDs encoding the same function can have very different numbers of decision nodes, but even fewer decision nodes need not imply cheaper *mapped* gates. One structure may expose common AND subexpressions, another may induce many NOT gates, and a third may admit more balanced evaluation. Thus variable ordering and mapping must be evaluated *after* lowering into the actual library.

5.2 Influence-based variable ordering

A natural way to choose a decision variable is to ask whether changing it frequently changes the output. For a single Boolean function,

$$I_i(f) = \frac{1}{2^{n-1}} \sum_{x_{-i}} \mathbf{1}\{f(x_{-i}, 0) \neq f(x_{-i}, 1)\}. \tag{5}$$

A variable with large influence is frequently decisive. The heuristic places such variables earlier in one candidate order. This is a plausible structural guide, *not* an optimality theorem: sensitivity and BDD node count are different notions, and neither captures weighted gate area by itself.

Compute influence instead of taking it on faith. In the running example, f_1 chooses c when $a = 0$ and b when $a = 1$. Flipping a changes the output precisely when $b \neq c$, which occurs on two of the four (b, c) assignments. Therefore $I_a(f_1) = 1/2$. Flipping b changes f_1 when $a = 1$, while flipping c changes it when $a = 0$; so $I_b(f_1) = I_c(f_1) = 1/2$ as well. Influence cannot distinguish the selector from either data signal for this function. By contrast, $f_2 = b \vee (\neg a \wedge c)$ has $I_b(f_2) = 3/4$ and $I_a(f_2) = I_c(f_2) = 1/4$. The combined influence ordering can be useful but is inevitably sensitive to how multiple outputs are aggregated. It does not know the actual gate prices or the quality of downstream sharing.

5.3 Two-level Shannon expansion

One first-level decision may create a long chain of nested multiplexor-like logic. The next idea is to expand two levels at once, possibly choosing different second variables in the two branches. Formally,

$$\begin{aligned} f = {} & \neg x \wedge (\neg u \wedge f_{00} \vee u \wedge f_{01}) \\ & \vee x \wedge (\neg v \wedge f_{10} \vee v \wedge f_{11}). \end{aligned}$$

The derivation simply substitutes Shannon's identity into both cofactors. Its advantage is structural: four gated terms can often be computed in parallel and combined with a balanced OR structure, which may shorten the critical path. Its disadvantage is equally important: an expansion can duplicate shared subfunctions and inflate area. It belongs in a *portfolio*, not as an unconditional rewrite.

5.4 Small-support SOP and POS

The next route does not begin with a BDD at all. A variable is *essential* if its two cofactors differ. When a function depends only on a few essential variables, directly enumerate cubes (partial assignments), derive prime implicants, and select a greedy cover of the function's on-set. An SOP is exact if every selected cube implies the function and their union covers all true minterms. This is worth proving as two inclusions. Every selected cube lies inside the on-set, hence the SOP never outputs 1 where $f = 0$; coverage ensures the SOP never outputs 0 where $f = 1$. Consequently it agrees with f on the entire domain. A POS-like implementation can be obtained by covering the complement and negating its result. The optimization is incomplete because greedy cover is not guaranteed minimal, and prime enumeration can become exponential, so the implementation restricts support size. These are familiar issues in classical two-level logic minimization [4].

A cover on the running example. Since $f_2 = b \vee (\neg a \wedge c)$, the cube b covers all rows with $b = 1$, while $\neg a \wedge c$ covers the remaining true row with $a = 0, b = 0, c = 1$. Both cubes imply f_2, and together they cover its complete on-set: the expression is therefore exact, not merely a plausible simplification. Distributivity also yields the POS

$$f_2 = (b \vee \neg a) \wedge (b \vee c).$$

Neither canonical form is guaranteed to be cheapest after mapping: an AND2, OR2, or NOT may have different prices, and some of their subexpressions may already exist in another output's graph.

Example 2 (Factoring is a mapping choice)**.** The SOP $(a \wedge b) \vee (a \wedge c)$ is logically equal to $a \wedge (b \vee c)$. The first exposes two independent AND terms; the second shares a and may use fewer gates. Which has lower delay depends on $d_\wedge$, $d_\vee$ and the incoming arrivals. Thus algebraic equality is a correctness fact, while its desirability is a cost fact.

5.5 Portfolio selection and outcome

Round 2 compares a fixed collection of candidates built from alternative decision orders, two-level expansion and small-support implicant forms, then measures their exact final netlist costs. The WSL result has 25/25 PASS and 25/25 target delays, with summed area 790,243, a reduction of roughly 28.4% from Round 1. The search itself costs more runtime: median WSL runtime increases from 5.455 ms to 36.427 ms across those separately archived runs. Since these are not an interleaved pairing, the numbers indicate scale, not an exact causal speed ratio.

Decision after Round 2

Learning: representation diversification succeeds at closing the timing-feasibility gap. But much area remains, and an alternative can become more useful when its negated form or downstream slack is exploited. Next, look at the entire shared graph under case-specific prices and deadlines.

6 Round 3: account for complements, shared algebra and timing slack

6.1 Complemented edges: negation is information before it is a gate

If $g = \neg f$, it seems wasteful to store two unrelated Boolean subgraphs representing f and g. A complemented-edge BDD reference stores a regular node identity plus one inversion bit. The graph can share the same structural nodes, while the inversion bit carries the semantic complement. To prevent multiple redundant encodings, the unique table normalizes one child-complement convention before interning nodes. This is a representation optimization; when mapped into physical AND2/OR2/NOT, a required inverter must still be emitted or shared and charged.

An affine detector also recognizes functions $c \oplus \bigoplus_{i \in S} x_i$. If the high cofactor equals the negation of the low cofactor, Shannon decomposition reduces to XOR with the decision variable. To check the algebra, write $q = f|_{x=0}$ and $f|_{x=1} = \neg q$; then

$$f = (\neg x \wedge q) \vee (x \wedge \neg q) = x \oplus q.$$

This recognizes one XOR-shaped decision, not automatically an arbitrary global affine function; full affine recognition requires the appropriate recursive consistency across the remaining variables. It can allow structured balanced XOR constructions, all lowered through legal gates. It can help parity-like truth tables that ordinary BDD mapping handles awkwardly.

6.2 Why timing slack changes the optimization problem

Imagine a network whose target delay is 30 but whose actual delay to an output is 22. A gate-level rewrite that increases an intermediate arrival by 2 is not automatically harmful if every downstream path still finishes by 30. A greedy "never increase any arrival" policy leaves potentially cheaper gates unexplored.

The more principled tool is *required time*. Assign $R(y) = D_{\text{target}}$ at each primary output and propagate deadlines backward:

$$R(u) \leftarrow \min\{R(u), R(v) - d(v)\} \quad (u \in \text{fanin}(v)). \tag{6}$$

A shared node receives the strictest requirement over its consumers. Its slack is $S(u) = R(u) - T(u)$. We can then consider a cheaper De Morgan mapping even if it slightly increases local delay, provided the new arrival does not exceed $R(u)$. For instance,

$$a \vee b = \neg(\neg a \wedge \neg b) \tag{7}$$

may substitute a cheap AND and shared inversions for an expensive OR. An isolated sufficient area comparison involves $a_\vee$ versus $a_\wedge + 3a_\neg$, but the actual DAG cost depends on which inversions are already present and which gates stay alive due to fanout. Global remeasurement is essential.

Lemma 3 (Fixed-boundary timing invariant)**.** *Start from a feasible DAG G and compute the old required times R_G backward. Replace an internal cone by an equivalent acyclic cone with the same external boundary. If every replacement root exposed to the unchanged graph has newly computed arrival at most its old required time, and the unchanged gates preserve their old types, delays and dependency edges, the new output delay is at most the target.*

Proof. Contract each new cone into its exposed root. Each such root meets the old required bound by assumption. For an unchanged gate v, each incoming unchanged dependency $u \to v$ obeys $R_G(u) \le R_G(v) - d(v)$. Thus by topological induction $T'(v) \le \max_u R_G(u) + d(v) \le R_G(v)$, including unary NOT with one predecessor. Every output requirement remains at most the original delay target; an output reused internally may have a strictly smaller required time. □

Why the assumptions matter. A rewrite that changes fanout, creates a new cross-cone dependency or reuses stale required times after another transformation is outside this lemma. The implementation remeasures the whole candidate DAG, and the accepted-candidate ranking provides a second engineering safeguard. A sufficient condition is not a proof of optimal area or of correctness of arbitrary rewrite code.

6.3 Balancing, bounded search and outcome

Associative AND/OR chains are re-parenthesized when doing so does not break essential sharing, and additional variable orders are attempted under a computation budget. Every candidate is pruned and evaluated as a complete live DAG. The archived Round 3 WSL Area is 533,705 with 25/25 PASS and 25/25 target delay; this is approximately 32.5% below Round 2. The magnitude is a central Stage 1 result, but it is the combined effect of several features. It should not be attributed solely to required-time recovery without the corresponding ablation data.

6.4 What the archived Round 3 ablation actually establishes

We later recovered a development summary that was missing from the first writing draft. Rather than treating the final result as one opaque improvement, it lists concrete snapshots along an algorithmic path. Starting at the 790,243-area portfolio, the successive archived snapshots record 777,324 after complemented-BDD development, 770,195 after affine detection, 762,164 after balancing, 586,218 after slack-aware recovery, 543,209 after an additional reverse-order candidate, and 533,705 after legacy fallback selection. All listed entries have 25 successful target checks in their respective evaluations.

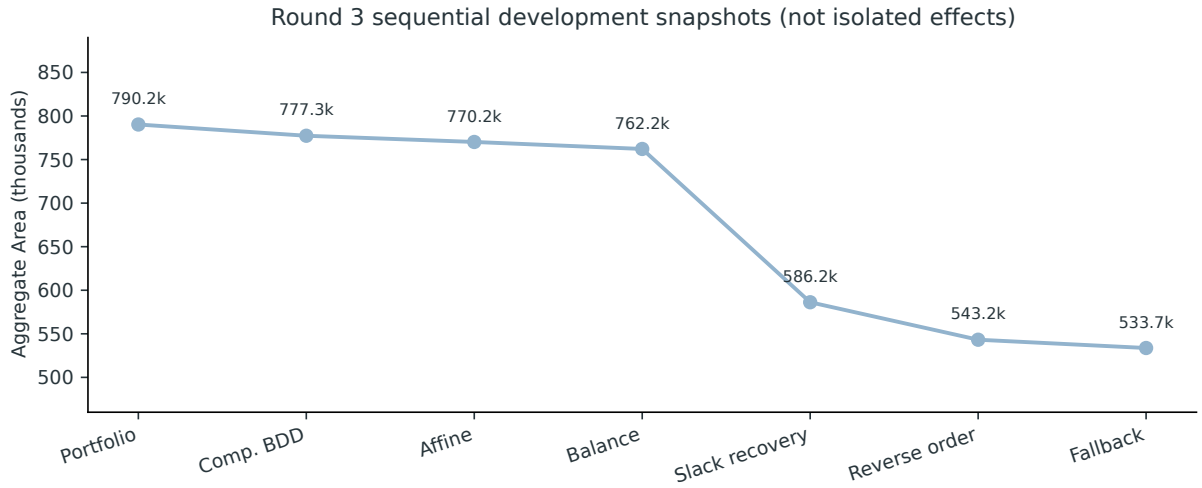


Figure 1: The chronological Round 3 *development path*; the separate implication experiment in the source CSV is deliberately not connected as though it were the next sequential stage.

Why does the distinction matter? The 175,946 difference between the balanced snapshot and the slack-recovery snapshot is an observation after a sequence of earlier changes. It illustrates why required-time reasoning was so promising, but it does not prove that activating the recovery routine on every possible prior configuration saves exactly that amount. A controlled factorial study would require frozen alternatives differing in only one feature, compiled and evaluated under matched conditions. Our source supports a strong *development narrative*, not that stronger causal claim.

Decision after Round 3

Learning: exact Boolean identities can be used more effectively when the optimization considers shared nodes and global deadline slack. Remaining gate-level opportunities may require selecting positive or negative representations across *several* fanouts. Next, explore polarity and algebraic restructuring.

7 Round 4: phase selection and algebraic rewriting

7.1 The missing global choice

A physical inverter can be shared among several consumers. Therefore asking "Should this OR gate be replaced by De Morgan?" independently at each gate misses a global decision: which *phase* of each shared signal should be stored? Define for node v the phase bit $p_v \in \{0, 1\}$ and stored function

$$g_v = \neg^{p_v} f_v. \tag{8}$$

Changing p_v switches whether its children must be complemented and whether a logical AND is physically realized as an AND or, under De Morgan, as an OR on the opposite phases. For example, if a shared signal $z = u \vee v$ is stored negatively as $\neg z = \neg u \wedge \neg v$, an AND2 may replace an OR2 but only if the required child polarities can be produced at acceptable cost. No NOT gate is "free" merely because the internal BDD reference has an inversion bit. A shared NOT is paid once per realized source polarity, not once per fanout. We evaluate a surrogate phase objective and use coordinate descent as a heuristic; exact output DAG area and delay still decide whether a candidate wins. This is not a global minimum-area solver.

A second pass applies absorption and factoring across shallow gate patterns. Its gains can be small because the original Boolean structure has already been aggressively simplified. It must also avoid an incorrect inference: if one rewrites a local cone to use fewer gates, the entire netlist may remain the same size when its allegedly removed ancestors are still referenced elsewhere.

7.2 Observed result and the search-cost warning

Round 4 reaches summed Stage 1 area 528,910, down from Round 3 by 4,795 (approximately 0.90%), with all 25 targets still met. But the archived WSL median synthesis time is 127.568 ms, compared with 68.486 ms for Round 3. The comparison is historical rather than same-run paired, yet the cost increase is sufficiently concerning to motivate a dedicated runtime investigation. A separate exploratory variable-order search occasionally expanded the BDD dramatically; candidate limits and fallback are needed to prevent unbounded resource consumption.

7.3 Recovering the missing postprocessor experiment

The newly located `postprocess_comparison.csv` helps answer the question: was it the De Morgan phase pass or algebraic rewriting that mattered more? Applied separately to the archived Round 3 circuits, algebra only reduces total area from 533,705 to 532,416, while phase only reaches 533,378. The two orders of composition are slightly different: phase followed by algebra gives 532,078, and algebra followed by phase gives 532,080. Taking the best strategy for each of the 25 cases produces 532,070.

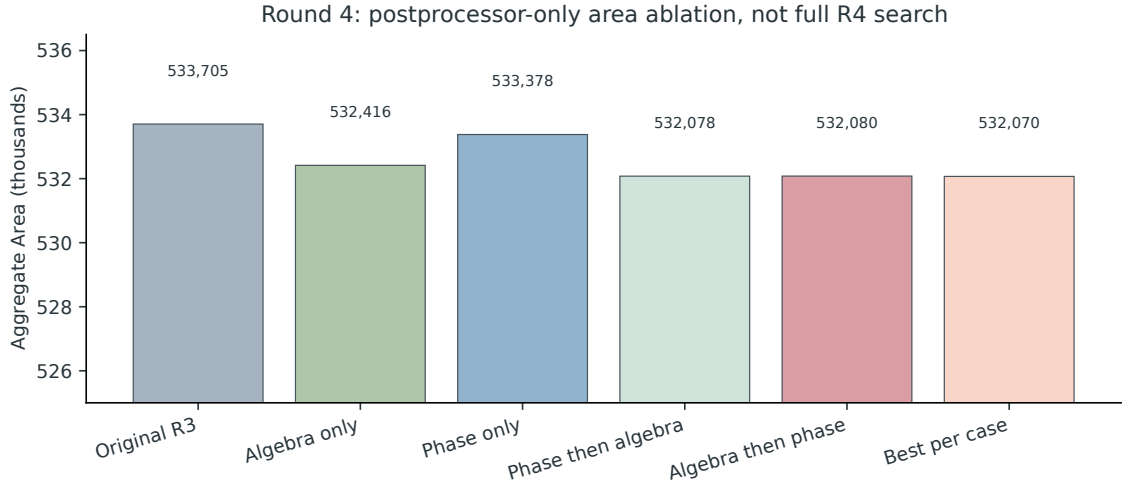


Figure 2: Round 4 postprocessing ablation. These treatments reuse the same original circuits; extra variable-order search is intentionally not included.

The important insight is not that one rewriting order always wins. Both the circuit's shared fanout and the local AND/OR/NOT prices determine which candidate is best. The 532,070 total uses a *per-case selection*, which makes

sense for a multi-candidate synthesizer but should not be confused with a single rewrite rule. The fully fledged Round 4 result of 528,910 also contains further variable-order exploration. Thus the difference between 532,070 and 528,910 cannot be counted as a benefit of algebra and phase postprocessing alone.

Decision after Round 4

Learning: increasing the candidate pool is not free. A roughly one-percent Area gain can be purchased with disproportionately more computation. Instead of making the Boolean search even more exhaustive, first ask what work the program repeats unnecessarily.

8 Round 4.1: construct the first explicit Area–Runtime Pareto comparison

Counterfactual considered. We could continue adding variable orders or polarity restarts and keep every tiny public-case area improvement. But this would spend more of a finite synthesis budget on strategies whose extra benefit had become small. The alternative was not to abandon the improved R4 circuit: it was to ask how much area a faster search would actually sacrifice.

The next experiment varies the amount of polarity postprocessing and variable-order exploration. It retains three executable configurations rather than presenting one parameter setting as universally best: **Strict** minimizes perturbation of the R4 circuit, **Balanced** accepts a tiny area change for runtime reduction, and **Fast** reduces the search further. These are meaningful *Pareto alternatives*: lower runtime can require worse area, and a global criterion is impossible without knowing the actual scoring function.

Two WSL measurements report that the Balanced configuration produces Area 529,014 in both runs; its median runtime is 96.048 and 102.615 ms. Strict preserves Area 528,910, while Fast reaches Area 530,822. In the case-level data, Fast has a conspicuous area regression in `case_018`; the small relative area-sum penalty conceals a large localized change. This demonstrates why we record per-case data and not only aggregate sums.

What makes two settings genuinely incomparable? Say configuration X dominates Y if $A_X \le A_Y$ and $T_X \le T_Y$, with at least one strict inequality, under matched evaluations. If X is smaller but slower, neither dominates the other: there is no single unambiguous winner without a scoring utility or an explicit time constraint. Strict and Balanced illustrate exactly this tension. This is a *Pareto* argument, not a claim that area and milliseconds can be added together in a physically meaningful unit.

Research decision: Balanced becomes the candidate for engineering optimization, not because its gate area is absolutely smallest, but because its area sacrifice relative to Strict is 104 out of roughly 529,000 while lowering median runtime. We retain Strict as the area-oriented comparator.

9 Round 5: remove recomputation before reducing search quality

The change of object. Until now, the algorithm had treated the circuit as the thing being optimized. Here the *synthesis program* becomes the object of analysis. This is not a semantic change to a Boolean function: we are asking whether repeated work inside the compiler can be removed before sacrificing any useful gate mapping candidate.

9.1 A useful conceptual split

There are two ways to accelerate a synthesis engine: *delete candidate searches*, or *implement the same searches more efficiently*. The latter is preferable when we already trust the candidate quality. Profiling revealed three repeated operations: multiple mapping policies rebuild the same ordered BDD; several small-support candidates recompute identical implicant covers; and every candidate's textual netlist is reparsed for area and timing estimates.

Fix the truth table F and variable order π. Let $B(F, \pi)$ be its immutable BDD. If policies $\theta_1, \dots, \theta_k$ share π, then they can all use the same BDD:

$$C_i = \mathrm{Map}(B(F, \pi), \theta_i). \tag{9}$$

The semantic reason is simple: BDD building depends on (F, π), not on how a later gate mapper chooses to realize its nodes. The cache key is logically meaningful: it must distinguish different truth functions and different variable orders, and the cached graph must not be modified by downstream candidates. If the mapper mutates the shared structure, reuse becomes a correctness risk rather than a safe speedup. Likewise, cache hits justify avoiding computation, not inventing a better Boolean function. Throughout this log we use $\wedge$, $\vee$ and $\neg$ for Boolean connectives; AND2, OR2 and NOT denote gate types in mathematical expressions; and the corresponding monospaced tokens are reserved for literal netlist syntax. In a simplified cost model the repeated build term decreases from kT_B to T_B. Crucially, this does *not* imply a

k-fold overall speedup when mapping or postprocessing dominates. Cached implicant covers follow the same logic: symbolic cube selections depend on the function and deterministic tie-breaking, while physical gate emission remains candidate-specific.

Round 5 also removes one provably duplicate candidate when $n > 13$. In the frozen configuration the pair differed only by an implicant mode that is disabled for those dimensions, so the two candidate computations were identical. A lightweight line scanner further reduces repeated netlist-statistic overhead.

9.2 What the paired evidence shows

In the dedicated Round 5 WSL pairing, Round 4.1 Balanced had median synthesis runtime 87.041 ms and Round 5 had 68.861 ms, both with Area 529,014 and 25/25 correct and target-met cases. All 25 emitted netlist files in that local comparison were byte-identical. These observations support a genuine implementation-efficiency gain. However, time-based candidate cutoffs mean the same byte identity is not guaranteed under every possible timing environment; one independent container rerun had a different yet valid candidate near a deadline.

Decision after Round 5

Learning: caches remove repeated *Boolean analysis*, but the engine still converts graphs to text and back as it rebalances and recovers area. If gate structure is already available, why throw it away between stages? This directly motivates typed in-memory DAGs.

10 Round 5.1: treat the netlist as a data structure, not repeated text

Why another implementation round? Caching helped with BDDs and covers, but the candidates still passed through several text-to-graph and graph-to-text transformations. Those conversions have no Boolean significance; they exist only because of an inconvenient intermediate representation. Round 5.1 therefore tests whether a typed internal graph can retain the same transformation logic while making it cheaper to execute.

10.1 The engineering hypothesis

A text format is a good external interface but a poor internal representation for repeated graph operations. Instead of parsing a netlist for rebalancing, serializing it, then parsing again for area recovery, Round 5.1 stores a typed gate table $v \mapsto (\mathrm{op}, a, b)$ and an explicit output list. Structural hashing interns identical immutable gates; reachability pruning discards dead nodes at the final serialization step. For commutative AND2/OR2 nodes the builder sorts input IDs before hash lookup. For rebalancing, a same-operation internal cone is flattened only when the interior nodes have a single consumer, while output references are included in the fanout count. An interior gate used by two consumers cannot simply be dissolved into one of them: the other consumer still needs that signal. In graph terms, only *unshared* associative interiors can be freely re-parenthesized without requiring a separate replacement for the other use. This matters in a multi-output network: reusing an output as someone else's intermediate result must not cause that shared computation to be silently removed. A topological induction over gate IDs proves the functional equivalence of these associative, commutative and idempotent rewrites; a reverse reachability pass from *all* output roots preserves every live shared gate.

If a new gate is created only after its fanins exist, its arrival can be stored immediately:

$$T(v) = \begin{cases} T(a) + d_{\neg}, & \mathrm{op}(v) = \mathrm{NOT}, \\ \max\{T(a), T(b)\} + d_{\mathrm{op}(v)}, & \text{otherwise.} \end{cases} \tag{10}$$

Topological induction proves this stored arrival correct. This is a real *incremental arrival-time* computation, but the implementation does **not** claim incremental global area accounting. A replacement may free an entire private cone, but a gate that still serves another output remains live. Exact area is recomputed after reachability pruning to avoid charging errors.

Follow one gate through the builder. The input Netlist assigns every gate an ID greater than its fanin IDs. The in-memory parser stores that ordered graph as nodes (op, a, b); the builder can therefore compute $T(v)$ immediately after a new node is created. When two AND2 nodes receive the same two operands in opposite orders, sorting their IDs makes their hash keys identical, which is legal because $u \wedge v = v \wedge u$. The key does *not* unify arbitrary Boolean-equivalent but structurally different expressions: that would require more algebraic reasoning. After transformations, the serializer traces backward from *every* output, discards nodes outside the union of their ancestor sets, and renumbers the rest.

Induction over this topological order explains both semantic preservation and why a gate feeding multiple outputs must be counted only once.

Limits of the proof. This reasoning assumes each local rewrite implements the intended Boolean identity and that its new dependencies remain acyclic. The actual program still needs parser validation, full-DAG rescoring and independent netlist checks; a correct identity on paper cannot rule out an indexing bug in C++.

10.2 Why this is not just another heuristic

Round 5.1 keeps the same mapping policies and internal cost ranking as Round 5. The typed DAG covers the high-frequency candidate balancing/recovery passes, but the final phase/algebraic alternatives still serialize and reparse netlists; the system is not entirely text-free. The aim is to change the *cost of executing* the optimization, not its mathematical candidate space. Accordingly, functional identity and equal output netlists are strong regression checks. In a first WSL paired run all 25 Round 5 and 5.1 outputs were byte-identical; the second had the same per-case areas and delays. Under a clock-bounded search, this remains an empirical property rather than a cross-platform guarantee. In particular, the source uses elapsed-time checks before later modern and legacy candidates, additional postprocessor eligibility thresholds, and periodic time checks for selected BDD orders; none constitutes a hard preemption mechanism for all stages. Since the candidate set is timing-dependent, two correct executions can output different valid netlists while using the same lexicographic ranking. The comparison to Round 5.1 therefore supports an engineering-efficiency claim, not a deterministic-output or optimality theorem.

10.3 Measured speed and generalization

The WSL paired medians change from 72.188 to 55.488 ms in one run and 60.827 to 55.077 ms in the other. Both versions have 25/25 PASS, 25/25 target delays and total Area 529,014. The average of the two per-run medians improves from 66.508 to 55.283 ms, about 16.9% (a descriptive two-run estimate). A mixed holdout of 80 functions and a 16-case larger set also pass the evaluator and independent checker, but the gains are distribution-dependent: small cases may see little benefit or small overhead, while larger graphs profit more from avoided parsing.

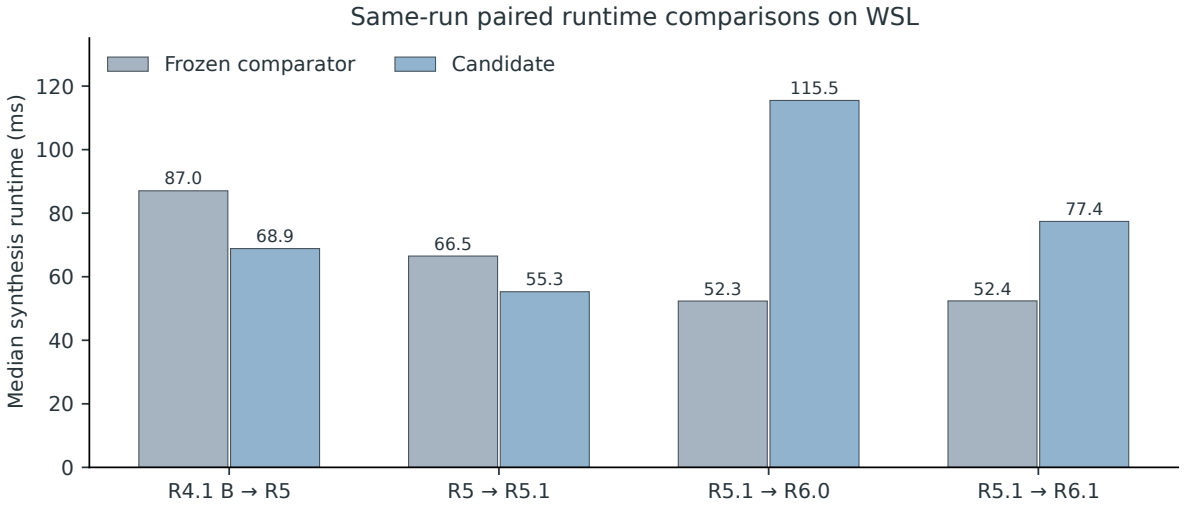


Figure 3: Paired WSL comparisons from distinct recorded experiments. The figure is constructed from authentic CSV records with the bundled plotting script; inter-group medians are not interchangeable experiments.

Decision after Round 5.1

Learning: the engineering refactor achieves another measurable runtime gain without sacrificing measured gate quality. The next research question should not be "how many more milliseconds can be shaved?" but "is the chosen function decomposition leaving significant circuit-area opportunities on the table?" Freeze this implementation as the efficiency baseline.

10.4 A limited memory snapshot: what Peak RSS says

After the source freeze was checked on the student's own WSL installation, we ran the exact research source on the large `case_024` as a standalone process under GNU `/usr/bin/time -v`. The terminal reported normal exit, wall-clock

time 1.66 s and peak resident memory 69,480 KiB (67.85 MiB); the synthesizer itself reported area 124,475 and delay 70. The raw `/tmp` time log was not uploaded, so this writing archive labels the record as a student-supplied terminal transcription.

The measurement answers a narrow question: *how much memory did one particular process run make resident at its peak?* It does not bound worst-case memory across the 25 official cases, does not measure all virtual-memory allocations, and must not replace the official evaluator's per-case synthesis runtime. Preserving these distinctions is part of experimental reproducibility, not a cosmetic disclaimer.

11 Round 6.0: a change of representation, not merely another BDD order

Why resume Boolean optimization after engineering speedups? A faster synthesis engine is valuable only if it gives room to seek better realizations, or if synthesis time itself is a scoring concern. The next experiment deliberately returns to gate quality: a mapped DAG can still contain locally expensive expressions that are not exposed by changing the original BDD order. This is a new hypothesis, not an extension of the caching theorem.

11.1 Motivation: escaped local equivalences

A BDD-derived circuit may already be compact as a BDD yet still contain local multi-gate patterns that another circuit can implement more cheaply. The new idea is to inspect *the final gate DAG itself*, choose a small boundary of signals, recover the exact Boolean subfunction between that boundary and an internal output, and resynthesize the subfunction independently of the original local shape.

11.2 What exactly is a feasible cut?

For a chosen node v, a cut C_v is a collection of upstream boundary nodes meeting every path backward from v to a primary input. A k-feasible cut has at most k leaves. Round 6.0 fixes $k \le 4$. Treat the cut-leaf values as *formal* independent variables solely when enumerating assignments. This is not probabilistic independence: the real leaf signals can be correlated, but any replacement equal on all 2^k formal assignments is certainly equal on the subset the surrounding circuit can actually produce.

With $k \le 4$, the local truth table uses at most $2^4 = 16$ bits:

$$M_{v,C} = \sum_{s=0}^{2^k-1} f_{v,C}(s)\, 2^s. \tag{11}$$

This small mask is the bridge between an arbitrary gate cone and generic Boolean synthesis; classical DAG-aware AIG rewriting offers a related but more comprehensive rewrite-library approach [5, 6]. Candidate realizations include SOP, complemented SOP/POS, shallow factoring and one-variable Shannon decompositions. All use AND2, OR2 and NOT. The search is *not* complete and should not be called an exact minimum-area optimizer.

11.3 Why gate sharing makes local estimates dangerous

A cut mask that can be checked by hand. Return to the example's second output $f_2 = b \vee (\neg a \wedge c)$. Treat (a, b, c) as formal cut leaves and number assignments by $s = 4a + 2b + c$. The rows where $f_2 = 1$ are $s = 1, 2, 3, 6, 7$, so its 8-bit truth-function mask is

$$M_{f_2,\{a,b,c\}} = 2^1 + 2^2 + 2^3 + 2^6 + 2^7 = 206 = \texttt{0xCE}.$$

A local resynthesizer can check candidate expressions against these eight bits *before* considering gate costs. Equality on all formal boundary assignments guarantees equality for the actual boundary signals, even if they are not independently realizable; the guarantee is sufficient, not necessarily a minimal test. It says nothing yet about cycles, the output interface, or area.

Count the surviving DAG, not the rewritten text. Let $L(G)$ be the set of gates reachable from *any* primary output. With prices $a(v)$, the exact area is $A(G) = \sum_{v \in L(G)} a(v)$. If a candidate "eliminates" a gate which still feeds another output, that gate remains in $L(G)$ and its area is not saved. Our running example shows this sharply: after replacing gate 8, gate 7 is no longer used by 8, but must remain because it is still the first output. The original and rewritten circuits both cost 11; the benefit is delay, not area. A valid local rewrite therefore needs both semantic equality and a complete live-DAG cost check.

Suppose a cut replacement seems to save a gate g. If another output still references g, deleting one use does not eliminate its physical cost. Conversely, a rewrite can free several upstream private nodes at once. The implementation

conservatively estimates removable gates through reference counts and counts the candidate local cone cost without treating speculative gates from rejected alternatives as free. Only after an actual candidate is produced does it prune and measure the complete graph, retaining it when true area drops and the deadline remains met.

11.4 Outcome: a successful hypothesis but costly implementation

In the user's two WSL paired runs, Round 6.0 area changes from 529,014 to 528,738 and 528,744, with 25/25 PASS and target delays. Median runtime, however, increases from 53.286 to 114.610 ms and from 51.403 to 116.351 ms. In another set of 40 generated functions, area falls from 9,709 to 9,496; in an older 80-case set it falls from 22,821 to 22,579; for 16 larger cases it falls from 157,944 to 157,440. These tests support that additional optimization opportunities exist beyond particular public case IDs, but the synthetic distributions do not prove hidden-case success.

Decision after Round 6.0

Learning: new area savings are real, but the cost of repeatedly enumerating cuts and rebuilding equivalent cover candidates is too high. The next question should be "can the same local search be made cheaper?", not "can we enlarge k?"

12 Round 6.1: a deliberate Go/No-Go experiment

12.1 Predeclared engineering constraint

We froze $k \leq 4$ and made the optimization budget explicit. The cut postprocessor receives 25 ms wall-clock budget, with a second constraint linked to the per-case runtime allowance, and keeps caps on retained cuts and visited roots. The baseline Round 5.1 candidate remains available as fallback. This is *not* a 25 ms bound on the whole synthesis process.

12.2 Reuse local Boolean work

For a four-leaf cut, a truth mask contains at most 16 bits, but the same function may appear at many locations. We cache symbolic implicant covers under

$$K = (k, M, p), \tag{12}$$

where k is the number of formal leaves, M the truth mask, and p the deterministic greedy cover policy. The cache stores no actual DAG gate IDs, no fanout, and no timing data. Thus a hit preserves the symbolic Boolean cover while each candidate is re-emitted against its own physical cut leaves. Canonicalizing arbitrary input permutations would require tracking the permutation; this implementation does not silently identify them.

Cuts are also prioritized using a conservative estimate of removable *private* old gates. Finally, the complete netlist is remeasured before an accepted rewrite can replace the incumbent. The method is designed to reduce wasted search, not to expand the mathematical search space.

12.3 What the local evaluation actually concluded

Two WSL comparisons reported Round 6.1 areas 528,854 and 528,839 versus 529,014, with full official PASS and target delay. Median synthesis runtime increased from 54.572 to 80.731 ms and from 50.224 to 74.083 ms. Averaging paired medians produces a roughly 47.7% increase, exceeding the previously selected 30% tolerance. The small area improvement, about 0.03% on the public case sum, is not enough to justify replacing Round 5.1 as default.

The fresh 40-case mixed and 8-case larger holdouts nonetheless demonstrate valid generalizing area savings: 11,186 to 10,858 and 67,275 to 67,047, respectively; all pass both correctness and target delay, but their synthesis medians rise from 7.852 to 10.762 ms and from 68.093 to 94.327 ms. The positive and negative conclusions must be presented *together*.

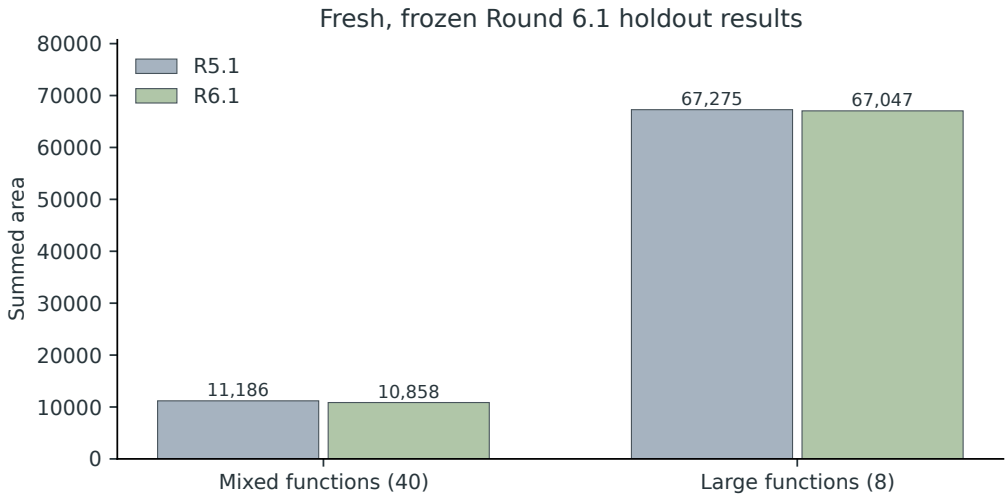


Figure 4: Fresh Round 6.1 holdout area by function family; source data and rendering script are included.

12.4 What the development budgets really measured

We also recovered two intermediate budget sweeps. On the 40-function mixed *development* set, the nominal cut budgets 25, 35 and 45 ms all save the same 213 local area units. The corresponding median *postprocessor* wall times are 4.84, 4.59 and 4.91 ms, a fluctuation too small to interpret as a monotone speed law. On a different, larger *development* set of 16 functions, increasing the budget produces 358, 408 and 501 area units saved, while median local wall time rises from 31.66 to 40.89 to 52.38 ms.

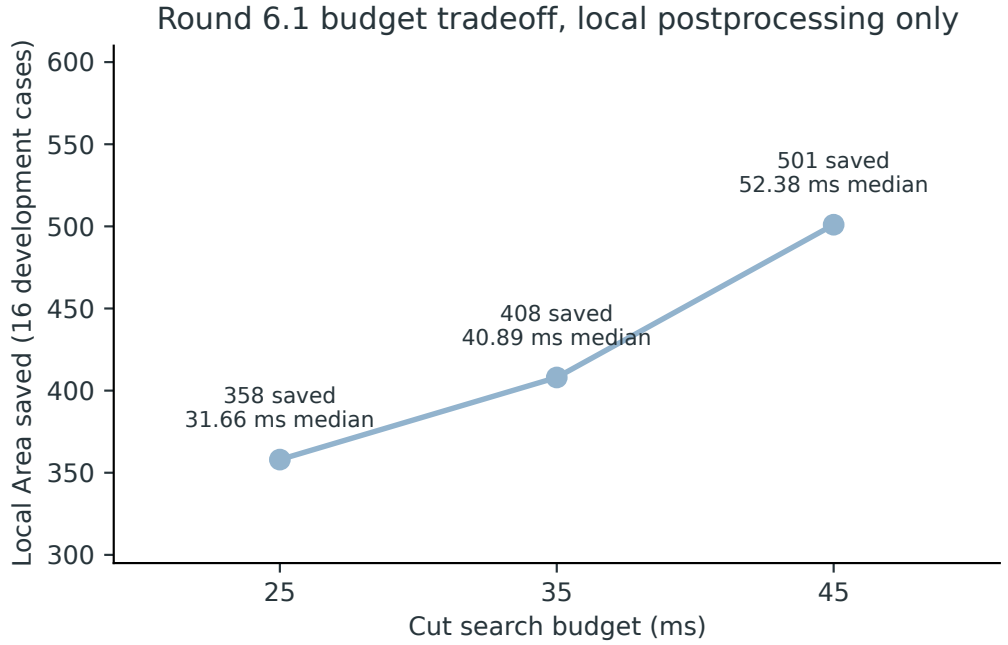


Figure 5: Larger-development-set Cut Budget experiment. “Area saved” measures the local postprocessor; these are not sums from the official Stage 1 evaluation.

Decision logic: on easy local structures, more budget may buy nothing; on harder structures, it can buy more rewrites but at an increasing cost. That explains why the 25 ms limit was a defensible engineering compromise. It does not imply that 25 ms is optimal for every unknown truth table. These were *development* sets already consulted during budget selection, so we cannot rebrand them as fresh blind holdouts.

Decision after Round 6.1: No-Go for automatic promotion

Learning: the hypothesis "cut rewriting can reduce area" is supported, while the stronger hypothesis "the current bounded rewriting implementation is worth its runtime" is rejected under our predeclared local criterion. The appropriate research response is to preserve the evidence and stop incremental threshold tuning, not to erase the unsuccessful branch.

13 How We Constructed Non-Official Test Cases

Motivation. Repeatedly modifying a synthesizer after checking the same 25 Stage 1 inputs risks learning incidental features of those particular Boolean tables. More importantly, the public set alone cannot tell us whether an optimization helps affine parity-like functions, mux-heavy structures, sparse products, shared outputs, or dense random truth tables equally. We therefore constructed additional functions from *generative descriptions* and only then expanded them into complete truth tables. This is not a new scoring system, and these cases are not professor-provided Stage 2 data.

Construction. A deterministic generator chooses a function family, a fixed pseudorandom seed, n input variables, m outputs, legal gate prices, and a delay target. For each assignment $x \in \{0,1\}^n$ it evaluates every output and serializes the resulting m-bit row. Thus the generator produces a legal complete instance, not a hand-selected Netlist. The mixed family includes affine functions (parity of selected inputs), sparse SOP, threshold functions, selector-based MUX, arithmetic bits, shared intermediate predicates, and dense deterministic truth bits. A separate large-function family composes local lookup tables, XORs, shared ensembles, and cascaded multiplexors. The scripts and case-hash manifests are preserved in `shared/test_generation/`; the original truth tables are retained separately rather than redistributed in a public writing package.

Evaluation protocol. The Round 5.1 test record has an 80-case mixed set and a 16-case large structured set, plus 110 older regression cases and two stress cases. Round 6.1 was additionally evaluated on a newly seeded 40-case mixed set and an 8-case large set after its budget was selected. In contrast, the 25/35/45-ms cut-search budget sweep was conducted on development sets; it should not be mislabeled an untouched holdout. Each run checks functionality, legality, target delay, area, and synthesis time against the appropriate frozen comparator. A deterministic seed makes a suite repeatable but does not make it representative of an unknown deployment distribution.

Research lesson. A regression suite answers *did this change break previously checked behavior?*; a frozen generated holdout asks *does the promising trend persist on a different specified family?*; only the later official Stage 2 release will answer the separate question of generalization to the next course distribution. These distinctions stop an attractive Stage 1 result from being exaggerated into a universal claim.

14 How did we guard against benchmark overfitting?

A public benchmark is a finite development set. Repeatedly adjusting thresholds until its aggregate area decreases risks overfitting the properties of those 25 released functions. We therefore distinguish *development evidence*, *repeated regression tests*, and *fresh generated holdouts* frozen after the candidate choices. Functions were generated from affine, sparse, threshold, multiplexing, arithmetic, dense and structured graph-related families across different sizes. Fresh holdout results were not used for further parameter tuning in the corresponding finalized experiments.

Nevertheless, there are limits. A synthetic generator creates its own distribution. A withheld seed is not equivalent to a future teacher-provided test distribution, and a collection already inspected during earlier iterations is no longer fully blind for later strategy decisions. We therefore claim only "no observed functional regression on the tested families" and, where supported, "area improvement on the fresh holdouts." We do not claim statistical proof of optimality or of performance on unreleased Stage 2 cases.

Independent correctness verification adds a different kind of reassurance. The official evaluator performs the required truth-table check under its contract, while a separately written checker validates legality and independently simulates circuits. For selected smaller functions the independent checker enumerates all rows; for larger ones it uses deterministic samples. The latter is not a formal universal proof, and the separate mathematical correctness arguments are still important.

15 A consolidated evidence narrative

15.1 The useful negative results

Not every valid rewrite should be adopted. In the present study, a large truth-table-derived candidate pool could become impractically expensive; phase/algebra refinements produced diminishing public-case area gains; the smaller generated

cases sometimes showed negligible runtime benefit from typed DAGs; and four-feasible cut rewriting reduced area but breached the local runtime-acceptance threshold. These observations are not defects to omit from a research record. Each separates a *correct mathematical transform* from a *justified engineering decision*. They also explain why further tuning of a public 25-case benchmark was stopped.

A particularly important counterfactual is the decision *not* to promote R6.1. Had we selected the default solely by public-case area sum, we would have ignored its roughly 47.7% local median-runtime penalty relative to R5.1. Had we dismissed it solely because of that penalty, we would have missed the independent evidence that cut rewriting can help other generated function families. Preserving the branch as an experimental result is more informative than either extreme.

15.2 Why the version history needs more than one number

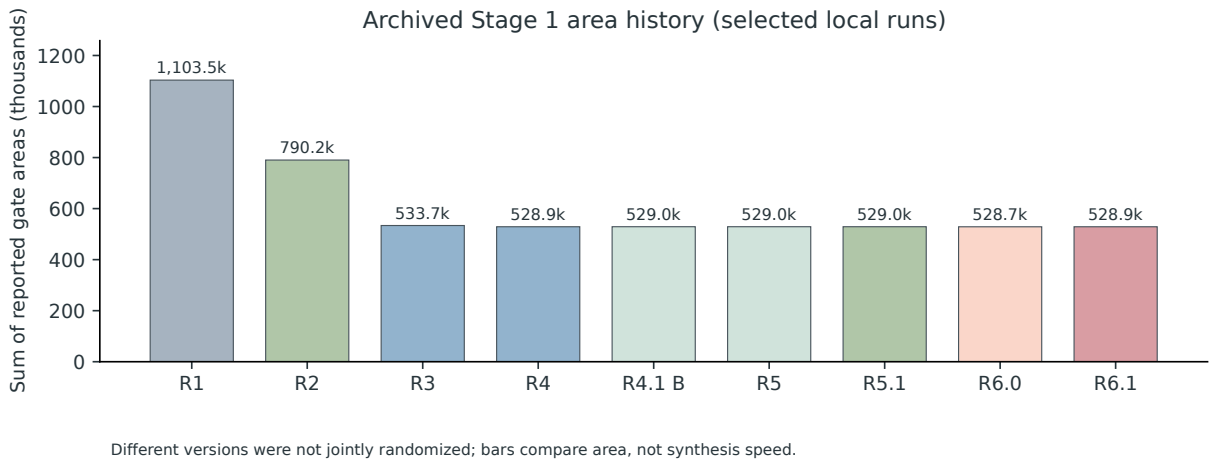


Figure 6: Selected archived local area totals. This is a non-paired development history; area totals are directly tabulated but runtime is not compared across archives.

Read the sequence by the *question solved*, not by the ordering of bar heights. R1 proves construction. R2 changes representations and meets the timing targets. R3 integrates the global deadline and reduces area substantially. R4 explores global phase and algebra, revealing expensive diminishing returns. R4.1 makes the Pareto tradeoff explicit. R5 and R5.1 refactor repeated computational work while preserving observed quality. R6.0 discovers an overlooked local area opportunity; R6.1 demonstrates the current technique's unfavorable cost under a fixed decision rule. These are complementary contributions, not a monotone sequence in all metrics.

Table 3: Local Stage 1 area milestones; a single designated run per round

Round	Target met	Summed area
1	12/25	1,103,455
2	25/25	790,243
3	25/25	533,705
4	25/25	528,910
4.1 Balanced	25/25	529,014
5	25/25	529,014
5.1 (default)	**25/25**	**529,014**
6.0 (prototype)	25/25	528,738
6.1 (research)	25/25	528,854

15.3 What was not proved

The work does not establish a globally minimum area or delay circuit for each input function. It does not show that lower delay after reaching target has zero score value; the numerical score formula was not published. A local gate-count reduction is not automatically a global area reduction in a shared DAG. BDD memory can be exponential. Synthesis time varies under CPU contention and wall-clock search cutoffs, so two valid runs can choose different candidates. These limitations should remain visible in any professional GitHub presentation.

16 How to reconstruct the evidence and figures

This writing archive includes all *intermediate datasets used by its charts*, not only final images. Run `python shared/scripts/make_figures.py` from the writing-package root (or call the script by absolute path); it reads the accompanying CSVs and the user-provided palette JSON. If the original local-results ZIP files are available, `python shared/scripts/extract_evidence.py -source-dir /path/to/zips` rebuilds the normalized data from the official CSV members; `verify_provenance.py` additionally checks exact archive digests and reconstructed output bytes. The SHA-256 archive provenance is retained in `source_manifest.csv`. The student-verified research freeze identifies `synth_round51.cpp`; its complete SHA-256 is in the bundled freeze manifest under `shared/data/raw_evidence/`; this is not an assertion of a final Canvas submission.

A reproducible public repository should preserve each round's source snapshot, compiler flags, evaluator commands, input manifest/hash, official CSV, independent verification result, candidate outputs, and document with the research decision. Large raw netlists can be archived separately, while the paper still includes enough information for a reader to reproduce its figures. The exact local output paths follow the project's convention `local_runs/YYYYMMDD_HHMMSS_Round*/report_ready/`.

16.1 A concrete replication exercise: from a saved official run to a plot

A plot is not primary evidence. For example, the historical area timeline was derived from nine selected official CSVs stored in *nine different local-run ZIPs*. Each row in the original CSV is an evaluator observation for one case, not an independently proved universal bound. To reconstruct the timeline properly, use the following sequence:

1. Identify the exact archive and CSV member in `shared/data/source_manifest.csv`. Check the SHA-256 of the complete original ZIP; an archive with the same filename but different bytes is not automatically the same experiment.

2. Extract the 25 case rows and require unique case IDs `case_001` through `case_025`, with explicitly parsed `status`, `target_met`, `area`, `delay`, and `runtime_ms`. A functionally valid PASS can have `target_met=no`.

3. Sum the 25 `area` values for the selected run. Compare the result to `round_history.csv`, then regenerate `area_evolution.pdf` using the packaged plotting script. This reproduces the *displayed result*, not the original synthesis computation.

4. To reproduce synthesis itself, obtain the separate frozen C++ source, official input truth tables and Linux x86 evaluator, rebuild with the recorded flags, and run a fresh evaluation. Retain the new logs and use the independent checker on the resulting netlists. Because later candidates can be gated by wall-clock time, a new valid run need not produce the identical network bytes.

The manuscript ZIP supplies the intermediate CSVs and plots; it *does not* supply the instructor-restricted truth tables or the complete original local-run ZIPs. The additional `verify_provenance.py` can audit only the first and third layers without original archives; passing that restricted audit is not evidence that the synthesis program itself was rerun.

16.2 How we checked that the evidence was not silently rewritten

For this review we ran the historical extraction program into an independent temporary directory. Fourteen normalized CSVs (nine selected official case tables, one run-history table, one manifest, one paired-benchmark table, one per-case comparison, and one holdout comparison) were byte-identical to the writing package. We also checked the standalone user-verified Round 5.1 source digest against the recorded manifest and the raw Round 3, Round 4 and Round 6.1 ablation input hashes. This verifies the lineage of those *particular files*; it does not prove that a human-entered cutoff was chosen before observing every development result, nor does it establish statistical representativeness of synthetic holdouts.

For public GitHub presentation, we should publish the generic scripts, mathematical explanations, citation records, curated aggregate results (subject to course permission), and exact source provenance. We should *not* assume permission to redistribute course-private full truth tables or the evaluator. These materials remain in private experiment archives, while the paper distinguishes available code from unavailable official inputs.

17 Reflection and next steps

Research lesson. A mathematical identity can certify equivalence; a cost model determines whether the identity is economically useful; a benchmark reports how a particular implementation behaved; and an adoption rule explains why that behavior warranted changing the default. Treating these as separate levels of reasoning is the central habit this

project developed. The largest early improvements came from changing the search representation and acknowledging global timing; the later rounds taught us when *not* to trade a large amount of computation for a tiny area gain.

The same small example clarifies what changes at each layer: a cofactor is a function, a BDD edge is a structural reference, a gate is a charged operation, and a path is a timed dependency. Reusing a reference, eliminating a live gate, and shortening a critical path are not interchangeable. This progression—Boolean function, shared decision graph, priced gate DAG, constrained timing graph, and resource-consuming search program—is the conceptual thread connecting the rounds.

An area-optimization algorithm can be mathematically sound and still be the wrong engineering choice when its marginal improvement is dwarfed by synthesis time. Preserving that negative result demonstrates an appropriate stopping criterion, especially under a rubric that rewards deep understanding and justified conceptual application. We therefore freeze Round 5.1 as the principal Stage 1 engineering baseline and preserve Rounds 6.0/6.1 as exploration branches, subject to the students' final submission choice.

For the later Stage 2 release, we should not reuse the published Stage 1 cases as though they were new holdouts. The actual submitted Stage 1 source should first be frozen, then both the old and any improved algorithm can be evaluated on the new release and on the original cases under matched conditions. This future work is outside the current Stage 1 record.

18 Collaboration, AI-Assisted Development, and Responsibility

18.1 How the two students collaborated

The two named student authors, Jincan LI and Zeyun DU, initially investigated approaches independently. They then compared ideas and integrated selected methods into a common research direction. This process was not divided into a fixed checklist of per-person ownership for BDDs, area recovery, or experimentation. We therefore describe the collaboration as independent exploration followed by mutual review and integration, without claiming that a specific individual authored each C++ module or mathematical derivation.

18.2 What AI actually did

Generative AI tools, principally ChatGPT, played a substantial role throughout the work. In particular, AI produced or substantially revised significant C++ implementation code, developed scripts for case execution, benchmarking, data gathering, verification and result packaging, and assisted in mathematical explanation, design suggestions, interpretation of measurements, and the preparation and iterative editing of these documents. This was not merely grammar correction, boilerplate generation, or occasional syntax help. The research record deliberately retains the generated tools and the results they produced rather than retroactively portraying the workflow as manual-only development.

18.3 What the human review and local evidence contribute

Human participation focused on asking which technical problem was worth solving next, questioning proposed optimizations and their marginal benefits, assessing the area–runtime trade-off, demanding checks against overfitting, and deciding when to keep or stop an approach. A student author independently executed the provided Linux evaluator and the local reproduction workflow in WSL, inspected the resulting summaries, and verified the chosen Round 5.1 source digest. These actions establish a concrete review and reproduction path; they do not support a claim that every line of code was written unaided or individually inspected in full.

Several risks follow directly from extensive AI assistance: an apparently plausible transformation may violate a graph invariant, a generated script may compare unmatched experimental runs, and a polished explanation may overstate what data establish. Our response was to retain source snapshots, evaluator outputs and independent checks, distinguish proofs from empirical findings, and revise claims whenever the evidence was narrower than an initial interpretation. This process improves traceability but cannot guarantee optimality or correctness on every unseen case. The authors remain responsible for the factual claims, appropriate citations, and final research decisions in any submitted or published version.

References

[1] UFUG2106 Course Staff, "Final project (tentative): Truth table to circuit," 2026, instructor-provided project specification and evaluator; distributed privately to students.

[2] C. E. Shannon, "The synthesis of two-terminal switching circuits," *Bell System Technical Journal*, vol. 28, no. 1, pp. 59–98, 1949.

[3] R. E. Bryant, "Graph-based algorithms for boolean function manipulation," *IEEE Transactions on Computers*, vol. C-35, no. 8, pp. 677–691, 1986, doi:10.1109/TC.1986.1676819.

[4] R. K. Brayton, G. D. Hachtel, C. T. McMullen, and A. L. Sangiovanni-Vincentelli, *Logic Minimization Algorithms for VLSI Synthesis*. Kluwer Academic Publishers, 1984, doi:10.1007/978-1-4613-2821-6.

[5] A. Mishchenko, S. Chatterjee, and R. K. Brayton, "DAG-Aware AIG Rewriting: A Fresh Look at Combinational Logic Synthesis," in *Proceedings of the 43rd Design Automation Conference (DAC)*, 2006, pp. 532–535, doi:10.1145/1146909.1147048.

[6] Berkeley Logic Synthesis and Verification Group, "ABC: A system for sequential synthesis and verification," https://github.com/berkeley-abc/abc, software reference; accessed November 2026.